

□ How to Use This Document

Run every query in this document yourself over the weekend before the project starts. This gives you a working database to compare against student work during monitoring. The Expected Result boxes tell you what numbers to look for when checking group outputs. Sections follow the same order as the student guides — Day 1 then Day 2.

DAY 1 — PART A: Schema & Staging Setup

A1 — Create Schema and Staging Table

```
CREATE SCHEMA IF NOT EXISTS tembo_hotel;
SET search_path TO tembo_hotel;

CREATE TABLE IF NOT EXISTS bookings_staging (
    booking_id          TEXT,  guest_name          TEXT,
    guest_phone         TEXT,  guest_city       TEXT,
    guest_nationality   TEXT,  room_no         TEXT,
    room_type           TEXT,  room_rate_per_night TEXT,
    check_in_date       TEXT,  check_out_date  TEXT,
    nights_stayed       TEXT,  staff_name      TEXT,
    staff_department    TEXT,  staff_salary    TEXT,
    payment_method      TEXT,  booking_status  TEXT,
    total_amount        TEXT,  service_used    TEXT,
    service_price       TEXT,  guest_rating    TEXT
);
```

A2 — Import CSV (pgAdmin Method)

Right-click **bookings_staging** → Import/Export Data → Format: csv → Header ON → select **tembo_hotel_dirty.csv** → OK

✓ Expected Result / What to Check:

SELECT COUNT(*) FROM bookings_staging; → should return 286
 SELECT * FROM bookings_staging LIMIT 5; → you will immediately see dirty names and dates

DAY 1 — PART B: Audit Queries & What You Will Find

□ Run these yourself first

These audit queries show the exact dirty problems in the dataset.

When monitoring groups on Monday, ask them to show you their audit results — they should match these.

Audit 1 — guest_name problems

```
SELECT DISTINCT guest_name FROM bookings_staging ORDER BY guest_name
LIMIT 40;
```

✓ Expected Result / What to Check:

You will see: 'ALICE MWANGI', 'alice mwangi', ' Carol Wanjiku ', 'Brian Otieno' etc.
Problems: UPPERCASE names, lowercase names, leading/trailing spaces
Count of problem rows: approximately 30-40 out of 286

Audit 2 — room_type distinct values

```
SELECT DISTINCT room_type, COUNT(*) AS count
FROM bookings_staging GROUP BY room_type ORDER BY room_type;
```

✓ Expected Result / What to Check:

Expected clean values: Standard, Deluxe, Suite, Penthouse (4 types)
Dirty values found: 'standard', 'deluxe', 'Std', 'DLX', 'STE' — approximately 7-8 distinct dirty values

Audit 3 — payment_method and booking_status

```
SELECT DISTINCT payment_method FROM bookings_staging;
SELECT DISTINCT booking_status FROM bookings_staging;
```

✓ Expected Result / What to Check:

payment_method dirty values: 'mpesa', 'CASH', 'card', 'bank transfer' (in addition to correct ones)
booking_status dirty values: 'checked out', 'CANCELLED' (should be title case)

Audit 4 — Date format problems

```
SELECT booking_id, check_in_date, check_out_date
FROM bookings_staging
WHERE check_in_date NOT SIMILAR TO '[0-9]{4}-[0-9]{2}-[0-9]{2}';
```

✓ Expected Result / What to Check:

You will see 3 date formats mixed in:
DD/MM/YYYY → e.g. '05/01/2024'
MM-DD-YYYY → e.g. '01-05-2024'
DD-MM-YY → e.g. '05-01-24'
Approximately 40-50 rows will have non-standard dates

Audit 5 — phone, city, salary, rating problems

```

-- NULL or empty phones
SELECT COUNT(*) FROM bookings_staging
WHERE guest_phone IS NULL OR TRIM(guest_phone) = '';

-- Phone format problems (+254 or dashes)
SELECT booking_id, guest_phone FROM bookings_staging
WHERE guest_phone LIKE '+254%' OR guest_phone LIKE '%-';

-- Salary stored as text with KES prefix
SELECT DISTINCT staff_salary FROM bookings_staging
WHERE staff_salary LIKE 'KES%';

-- Ratings out of valid range 1-5
SELECT booking_id, guest_name, guest_rating FROM bookings_staging
WHERE guest_rating NOT IN('1', '2', '3', '4', '5') OR TRIM(guest_rating) =
'';

-- Duplicate booking_ids
SELECT booking_id, COUNT(*) FROM bookings_staging
GROUP BY booking_id HAVING COUNT(*) > 1;

```

✓ Expected Result / What to Check:

NULL/empty phones: approximately 15 rows
 +254 prefix phones: approximately 14 rows
 Dashed phones (0712-345-678): approximately 14 rows
 KES salary rows: approximately 7 rows
 Invalid ratings (0, 6, or empty): approximately 10 rows
 Duplicate booking_id: BK0006 appears twice (1 exact duplicate)

DAY 1 — PART C: Complete Cleaning Script (Full Answers)

□ Instructor Use

This is the complete cleaning answer key. Students should work through this themselves. Only refer to specific fixes when a group has been stuck for 10+ minutes on a specific problem. Never hand them this document.

Clean 1 — guest_name: trim + fix casing

```
-- SELECT first
SELECT booking_id, guest_name, INITCAP(TRIM(guest_name)) AS fixed_name
FROM bookings_staging
WHERE guest_name != INITCAP(TRIM(guest_name));

-- UPDATE
UPDATE bookings_staging
SET guest_name = INITCAP(TRIM(guest_name))
WHERE guest_name != INITCAP(TRIM(guest_name));
```

Clean 2 — guest_phone: remove dashes, fix +254

```
-- Fix dashes: strip all non-numeric characters
UPDATE bookings_staging
SET guest_phone = REGEXP_REPLACE(guest_phone, '[^0-9]', '', 'g')
WHERE guest_phone LIKE '%-%';

-- Fix +254: remove all non-numeric, then prepend 0
UPDATE bookings_staging
SET guest_phone = '0' || SUBSTRING(REGEXP_REPLACE(guest_phone, '[^0-9]',
'', 'g'), 4)
WHERE guest_phone LIKE '+254%';

-- Set empty phones to NULL
UPDATE bookings_staging
SET guest_phone = NULL
WHERE TRIM(guest_phone) = '';
```

Clean 3 — guest_city: typos, casing, empty

```
-- Fix UPPERCASE cities
UPDATE bookings_staging
SET guest_city = INITCAP(TRIM(guest_city))
WHERE guest_city != INITCAP(TRIM(guest_city));

-- Fix typos: Nairobii → Nairobi, Mombasaa → Mombasa
UPDATE bookings_staging SET guest_city = 'Nairobi' WHERE guest_city ILIKE
'Nairobii';
```

```

UPDATE bookings_staging SET guest_city = 'Mombasa' WHERE guest_city ILIKE
'Mombasaa';
UPDATE bookings_staging SET guest_city = 'Kisumu' WHERE guest_city ILIKE
'Kisumux';

-- Set empty cities to Unknown
UPDATE bookings_staging
SET guest_city = 'Unknown'
WHERE TRIM(guest_city) = '' OR guest_city IS NULL;

```

Clean 4 — Date formats (3 formats to fix)

```

-- Fix DD/MM/YYYY (has forward slashes)
UPDATE bookings_staging
SET check_in_date = TO_DATE(check_in_date, 'DD/MM/YYYY')::TEXT,
    check_out_date = TO_DATE(check_out_date, 'DD/MM/YYYY')::TEXT
WHERE check_in_date LIKE '%/%';

-- Fix DD-MM-YY (2-digit year - ends in short year e.g. '05-01-24')
-- Identify: has dashes AND length = 8 characters
UPDATE bookings_staging
SET check_in_date = TO_DATE(check_in_date, 'DD-MM-YY')::TEXT,
    check_out_date = TO_DATE(check_out_date, 'DD-MM-YY')::TEXT
WHERE check_in_date LIKE '%-%'
    AND LENGTH(check_in_date) = 8;

-- Fix MM-DD-YYYY (has dashes, length=10, but month is FIRST)
-- These are tricky - look at the day part: if > 12 it MUST be a day not
a month
-- Safest: identify rows where SPLIT_PART gives impossible month > 12 in
DD position
UPDATE bookings_staging
SET check_in_date = TO_DATE(check_in_date, 'MM-DD-YYYY')::TEXT,
    check_out_date = TO_DATE(check_out_date, 'MM-DD-YYYY')::TEXT
WHERE check_in_date LIKE '%-%'
    AND LENGTH(check_in_date) = 10
    AND SPLIT_PART(check_in_date, '-', 1)::INTEGER <= 12
    AND SPLIT_PART(check_in_date, '-', 2)::INTEGER > 12;

```

Clean 5 — room_type: abbreviations and lowercase

```

UPDATE bookings_staging
SET room_type = CASE
    WHEN UPPER(TRIM(room_type)) IN ('STANDARD', 'STD') THEN 'Standard'
    WHEN UPPER(TRIM(room_type)) IN ('DELUXE', 'DLX') THEN 'Deluxe'
    WHEN UPPER(TRIM(room_type)) IN ('SUITE', 'STE') THEN 'Suite'
    WHEN UPPER(TRIM(room_type)) = 'PENTHOUSE' THEN 'Penthouse'
    ELSE room_type
END;

```

Clean 6 — payment_method and booking_status casing

```
UPDATE bookings_staging
SET payment_method = CASE
    WHEN UPPER(TRIM(payment_method)) IN ('MPESA', 'M-PESA', 'M PESA') THEN
        'M-Pesa'
    WHEN UPPER(TRIM(payment_method)) = 'CASH'
    THEN 'Cash'
    WHEN UPPER(TRIM(payment_method)) = 'CARD'
    THEN 'Card'
    WHEN UPPER(TRIM(payment_method)) IN ('BANK TRANSFER', 'BANK') THEN
        'Bank Transfer'
    ELSE payment_method
END;

UPDATE bookings_staging
SET booking_status = CASE
    WHEN UPPER(TRIM(booking_status)) = 'CHECKED OUT' THEN 'Checked Out'
    WHEN UPPER(TRIM(booking_status)) = 'CANCELLED' THEN 'Cancelled'
    WHEN UPPER(TRIM(booking_status)) = 'NO SHOW' THEN 'No Show'
    ELSE booking_status
END;
```

Clean 7 — total_amount and staff_salary (strip KES/commas, cast to number)

```
UPDATE bookings_staging
SET total_amount = REGEXP_REPLACE(total_amount, '[^0-9.]', '', 'g')
WHERE total_amount SIMILAR TO '%[^0-9.]%';

UPDATE bookings_staging
SET staff_salary = REGEXP_REPLACE(staff_salary, '[^0-9.]', '', 'g')
WHERE staff_salary SIMILAR TO '%[^0-9.]%';

-- Set empty total_amount to NULL
UPDATE bookings_staging SET total_amount = NULL
WHERE TRIM(total_amount) = '';
```

Clean 8 — guest_rating: invalid values → NULL

```
UPDATE bookings_staging
SET guest_rating = NULL
WHERE guest_rating NOT IN ('1', '2', '3', '4', '5')
    OR TRIM(guest_rating) = '';
```

Clean 9 — Remove impossible rows (negative nights, checkout before checkin)

```
-- Flag rows with negative nights first
```

```

SELECT booking_id, check_in_date, check_out_date, nights_stayed
FROM bookings_staging
WHERE NULLIF(REGEXP_REPLACE(nights_stayed, '^[^0-9-]', '', 'g'), '')::INTEGER
< 1;

-- These rows will be excluded during INSERT to production table (see
Clean 10)
-- Optional: delete them from staging now
DELETE FROM bookings_staging
WHERE NULLIF(REGEXP_REPLACE(nights_stayed, '^[^0-9-]', '', 'g'), '')::INTEGER
< 1;

```

Clean 10 — Remove exact duplicate rows

```

-- Confirm duplicates first
SELECT booking_id, COUNT(*) FROM bookings_staging
GROUP BY booking_id HAVING COUNT(*) > 1;

-- Delete keeping lowest ctid (first physical row)
DELETE FROM bookings_staging
WHERE ctid NOT IN (
    SELECT MIN(ctid) FROM bookings_staging GROUP BY booking_id
);

```

Clean 11 — guest_nationality casing

```

UPDATE bookings_staging
SET guest_nationality = INITCAP(TRIM(guest_nationality))
WHERE guest_nationality != INITCAP(TRIM(guest_nationality));

```

Create production table and load clean data

```

CREATE TABLE IF NOT EXISTS bookings (
    booking_id          VARCHAR(10) PRIMARY KEY,
    guest_name          VARCHAR(100), guest_phone          VARCHAR(15),
    guest_city          VARCHAR(60), guest_nationality      VARCHAR(60),
    room_no             VARCHAR(5), room_type              VARCHAR(20),
    room_rate_per_night NUMERIC(10,2), check_in_date        DATE,
    check_out_date      DATE, nights_stayed               INTEGER,
    staff_name          VARCHAR(100), staff_department     VARCHAR(50),
    staff_salary        NUMERIC(10,2), payment_method      VARCHAR(20),
    booking_status      VARCHAR(20), total_amount          NUMERIC(12,2),
    service_used        VARCHAR(60), service_price         NUMERIC(10,2),
    guest_rating        INTEGER
);

INSERT INTO bookings
SELECT
    booking_id, TRIM(guest_name),
    NULLIF(TRIM(guest_phone), ''),
    COALESCE(NULLIF(TRIM(guest_city), ''), 'Unknown'),
    INITCAP(TRIM(guest_nationality)),
    room_no, room_type,
    NULLIF(REGEXP_REPLACE(room_rate_per_night, '^[^0-9.]', '', 'g'), '')::NUMERIC,
    check_in_date::DATE, check_out_date::DATE,
    NULLIF(REGEXP_REPLACE(nights_stayed, '^[^0-9-]', '', 'g'), '')::INTEGER,
    TRIM(staff_name), TRIM(staff_department),
    NULLIF(REGEXP_REPLACE(staff_salary, '^[^0-9.]', '', 'g'), '')::NUMERIC,
    payment_method, booking_status,
    NULLIF(REGEXP_REPLACE(total_amount, '^[^0-9.]', '', 'g'), '')::NUMERIC,
    NULLIF(TRIM(service_used), ''),
    NULLIF(REGEXP_REPLACE(service_price, '^[^0-9-]', '', 'g'), '')::NUMERIC,
    NULLIF(guest_rating, '')::INTEGER
FROM bookings_staging
WHERE check_in_date SIMILAR TO '[0-9]{4}-[0-9]{2}-[0-9]{2}'
    AND check_out_date SIMILAR TO '[0-9]{4}-[0-9]{2}-[0-9]{2}'
    AND NULLIF(REGEXP_REPLACE(nights_stayed, '^[^0-9-]', '', 'g'), '')::INTEGER
> 0;

```

✔ Expected Result / What to Check:

SELECT COUNT(*) FROM bookings; → should return approximately 270–278 rows

Excluded rows: negative nights, impossible dates, exact duplicates

SELECT DISTINCT room_type FROM bookings; → should return exactly: Standard, Deluxe, Suite, Penthouse

SELECT DISTINCT payment_method FROM bookings; → exactly: M-Pesa, Cash, Card, Bank Transfer

SELECT DISTINCT booking_status FROM bookings; → exactly: Checked Out, Cancelled, No Show

Create v_clean_bookings view

```
CREATE OR REPLACE VIEW v_clean_bookings AS
SELECT *,
    DATE_TRUNC('month', check_in_date)           AS booking_month,
    TO_CHAR(check_in_date, 'YYYY-MM')           AS month_label,
    TO_CHAR(check_in_date, 'Month')              AS month_name,
    EXTRACT(YEAR FROM check_in_date)             AS booking_year,
    EXTRACT(DOW FROM check_in_date)             AS day_of_week_num,
    TO_CHAR(check_in_date, 'Day')               AS day_name,
    (nights_stayed * room_rate_per_night)       AS room_revenue,
    COALESCE(service_price, 0)                  AS service_revenue,
    CASE
        WHEN guest_rating BETWEEN 4 AND 5 THEN 'Happy'
        WHEN guest_rating = 3 THEN 'Neutral'
        WHEN guest_rating BETWEEN 1 AND 2 THEN 'Unhappy'
        ELSE 'No Rating'
    END AS sentiment
FROM bookings
WHERE booking_status = 'Checked Out';
```

Question 1A — Revenue by month

```

SELECT
    TO_CHAR(check_in_date, 'YYYY-MM') AS month,
    COUNT(*) AS bookings,
    SUM(total_amount) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_per_booking
FROM v_clean_bookings
GROUP BY TO_CHAR(check_in_date, 'YYYY-MM')
ORDER BY month;

```

✓ **Expected Result / What to Check:**

Expect 12-15 rows (one per month of 2023-2024 data)
 December 2024 and summer months (Jun-Aug) typically show highest revenue in this dataset
 Each month should show 15-25 bookings
 Total across all months should match: `SELECT SUM(total_amount) FROM v_clean_bookings`

Question 1B — Revenue by room type

```

SELECT
    room_type,
    COUNT(*) AS total_bookings,
    SUM(total_amount) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_revenue,
    ROUND(AVG(guest_rating), 2) AS avg_rating
FROM v_clean_bookings
GROUP BY room_type
ORDER BY total_revenue DESC;

```

✓ **Expected Result / What to Check:**

Penthouse: fewest bookings but highest avg revenue (KES 25,000+/night)
 Standard: most bookings, lowest rate — but may generate competitive total revenue
 Interesting insight: Standard rooms may generate more TOTAL revenue than Suite despite lower nightly rate
 Avg rating should be between 2.8 and 3.5 across all room types (randomised data)

Question 1D — Revenue pivot by payment method

```

SELECT
    TO_CHAR(check_in_date, 'YYYY-MM') AS month,
    SUM(CASE WHEN payment_method = 'M-Pesa' THEN total_amount ELSE 0 END)
AS mpesa_rev,
    SUM(CASE WHEN payment_method = 'Cash' THEN total_amount ELSE 0 END)
AS cash_rev,
    SUM(CASE WHEN payment_method = 'Card' THEN total_amount ELSE 0 END)
AS card_rev,

```

```

SUM(CASE WHEN payment_method = 'Bank Transfer' THEN total_amount ELSE
0 END) AS bank_rev
FROM v_clean_bookings
GROUP BY TO_CHAR(check_in_date, 'YYYY-MM')
ORDER BY month;

```

Question 3B — Guest satisfaction with happy % calculation

```

SELECT
    room_type,
    COUNT(guest_rating) AS
rated_count,
    ROUND(AVG(guest_rating), 2) AS
avg_rating,
    ROUND(SUM(CASE WHEN guest_rating BETWEEN 4 AND 5
        THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(guest_rating),
0), 1) AS happy_pct
FROM v_clean_bookings
GROUP BY room_type
ORDER BY avg_rating DESC;

```

Question 4C — Staff ranking overall + within department

```

WITH staff_totals AS (
    SELECT
        staff_name,
        staff_department,
        COUNT(*) AS total_bookings,
        SUM(total_amount) AS total_revenue,
        ROUND(AVG(guest_rating), 2) AS avg_rating
    FROM v_clean_bookings
    GROUP BY staff_name, staff_department
)
SELECT
    staff_name,
    staff_department,
    total_bookings,
    total_revenue,
    avg_rating,
    RANK() OVER (ORDER BY total_revenue DESC) AS overall_rank,
    RANK() OVER (PARTITION BY staff_department ORDER BY total_revenue
DESC) AS dept_rank
FROM staff_totals
ORDER BY overall_rank;

```

Question 5A — Month-over-month with CTE + LAG

```

WITH monthly AS (

```

```

SELECT
    TO_CHAR(check_in_date, 'YYYY-MM') AS month,
    SUM(total_amount) AS revenue
FROM v_clean_bookings
GROUP BY TO_CHAR(check_in_date, 'YYYY-MM')
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS
prev_month_revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS change_amount,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY month)) /
        NULLIF(LAG(revenue) OVER (ORDER BY month), 0) * 100, 1) AS
change_pct
FROM monthly
ORDER BY month;

```

✓ Expected Result / What to Check:

First row will have NULL for prev_month_revenue and change_amount (no prior month)
 Look for positive change_pct months = growth, negative = decline
 December typically shows large positive jump (holiday bookings in this dataset)
 Share this insight with groups: a negative MoM doesn't mean the hotel is failing — it depends on the season

Question 5B — Running total

```

WITH monthly AS (
    SELECT TO_CHAR(check_in_date, 'YYYY-MM') AS month, SUM(total_amount)
AS revenue
    FROM v_clean_bookings GROUP BY TO_CHAR(check_in_date, 'YYYY-MM')
)
SELECT month, revenue,
    SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue
FROM monthly ORDER BY month;

```

Question 6A — Cancellation rate overall

```

SELECT
    booking_status,
    COUNT(*) AS
count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct_of_total
FROM bookings
GROUP BY booking_status
ORDER BY count DESC;

```

✓ Expected Result / What to Check:

Checked Out: approximately 85-88% of all bookings

Cancelled: approximately 8-10%

No Show: approximately 4-6%

The SUM(COUNT(*)) OVER () trick gives the grand total without a subquery — ask groups if they understand it

F1 — v_monthly_revenue

```

CREATE OR REPLACE VIEW v_monthly_revenue AS
SELECT
    TO_CHAR(check_in_date, 'YYYY-MM') AS month,
    TO_CHAR(check_in_date, 'Month YYYY') AS month_label,
    COUNT(*) AS total_bookings,
    SUM(total_amount) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_booking_value,
    SUM(nights_stayed) AS total_nights
FROM v_clean_bookings
GROUP BY TO_CHAR(check_in_date, 'YYYY-MM'), TO_CHAR(check_in_date, 'Month
YYYY')
ORDER BY month;

```

F2 — v_room_performance

```

CREATE OR REPLACE VIEW v_room_performance AS
SELECT
    room_type,
    COUNT(*) AS total_bookings,
    SUM(nights_stayed) AS total_nights,
    ROUND(AVG(nights_stayed), 2) AS avg_nights,
    SUM(total_amount) AS total_revenue,
    ROUND(AVG(total_amount), 2) AS avg_revenue,
    ROUND(AVG(guest_rating), 2) AS avg_rating,
    ROUND(SUM(total_amount) / NULLIF(SUM(nights_stayed), 0), 2) AS
revenue_per_night
FROM v_clean_bookings
GROUP BY room_type
ORDER BY total_revenue DESC;

```

F3 — v_staff_performance

```

CREATE OR REPLACE VIEW v_staff_performance AS
SELECT
    staff_name,
    staff_department,
    COUNT(*) AS bookings_handled,
    SUM(total_amount) AS revenue_generated,
    ROUND(AVG(total_amount), 2) AS avg_booking_value,
    ROUND(AVG(guest_rating), 2) AS avg_guest_rating,
    RANK() OVER (ORDER BY SUM(total_amount) DESC) AS revenue_rank
FROM v_clean_bookings
GROUP BY staff_name, staff_department
ORDER BY revenue_generated DESC;

```

F4 — v_guest_insights

```
CREATE OR REPLACE VIEW v_guest_insights AS
SELECT
    guest_city,
    COUNT(*)                                AS total_bookings,
    COUNT(DISTINCT guest_name)              AS unique_guests,
    SUM(total_amount)                       AS total_revenue,
    ROUND(AVG(total_amount), 2)             AS avg_spend,
    ROUND(AVG(guest_rating), 2)             AS avg_rating,
    ROUND(AVG(nights_stayed), 2)            AS avg_nights
FROM v_clean_bookings
GROUP BY guest_city
ORDER BY total_revenue DESC;
```

F5 — v_cancellation_analysis

```
CREATE OR REPLACE VIEW v_cancellation_analysis AS
SELECT
    room_type,
    COUNT(*)
AS total_bookings,
    SUM(CASE WHEN booking_status = 'Checked Out' THEN 1 ELSE 0 END) AS
checked_out,
    SUM(CASE WHEN booking_status = 'Cancelled' THEN 1 ELSE 0 END) AS
cancelled,
    SUM(CASE WHEN booking_status = 'No Show' THEN 1 ELSE 0 END) AS
no_show,
    ROUND(SUM(CASE WHEN booking_status IN ('Cancelled', 'No Show')
        THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 1) AS
cancellation_rate_pct
FROM bookings
GROUP BY room_type
ORDER BY cancellation_rate_pct DESC;
```

F6 — Indexes

```
CREATE INDEX idx_bookings_checkin ON bookings (check_in_date);
CREATE INDEX idx_bookings_checkout ON bookings (check_out_date);
CREATE INDEX idx_bookings_roomtype ON bookings (room_type);
CREATE INDEX idx_bookings_status ON bookings (booking_status);
CREATE INDEX idx_bookings_staff ON bookings (staff_name);
CREATE INDEX idx_bookings_guestcity ON bookings (guest_city);
CREATE INDEX idx_bookings_payment ON bookings (payment_method);

-- Verify:
SELECT tablename, indexname FROM pg_indexes
WHERE schemaname = 'tembo_hotel' ORDER BY indexname;
```

✓ **Expected Result / What to Check:**

Should see 7 indexes created (plus the PRIMARY KEY index on booking_id = 8 total)

EXPLAIN on: SELECT * FROM bookings WHERE room_type = 'Deluxe';

Before index: Seq Scan on bookings

After index: Index Scan using idx_bookings_roomtype on bookings ✓

PART G — Expected Findings & Insights to Listen For

When groups present on Friday, these are the specific data-driven insights a strong group should mention. If a group gives vague statements like *'Standard rooms do well'*, push them with: *'Can you give me a specific number?'*

Area	Strong Finding (Specific)	Weak Finding (Vague)
Revenue	Penthouse generates ~KES X average per booking but only Y bookings/month. Standard rooms generate more TOTAL revenue despite lower rate due to higher volume.	'Penthouse is expensive'
Occupancy	Suite rooms have an average stay of X nights vs Standard at Y nights. Guests booking expensive rooms stay longer.	'Suites are popular'
Guests	Nairobi guests account for 60%+ of bookings but Eldoret guests have the highest avg spend at KES X.	'Most guests are from Nairobi'
Staff	Staff member [name] handles the most bookings but has the lowest avg rating of 2.X — this is a performance flag.	'Some staff are better than others'
Trends	Revenue grew from KES X in January to KES Y in August — a X% increase. December shows the highest single month.	'Revenue goes up in some months'
Cancellations	Suite rooms have the highest cancellation rate at X%. This may indicate pricing sensitivity at that level.	'There are some cancellations'

PART H — Quick Verification Queries (Run During Monitoring)

Use these quick queries to check a group's work without reading through all their code.

```
-- 1. Check their data is clean (run on their schema)
SELECT DISTINCT room_type FROM bookings; → must be exactly 4 clean values
SELECT DISTINCT payment_method FROM bookings; → must be exactly 4 clean values
SELECT DISTINCT booking_status FROM bookings; → must be exactly 3 clean values
SELECT COUNT(*) FROM bookings WHERE nights_stayed < 1; → must return 0
SELECT COUNT(*) FROM bookings WHERE check_out_date < check_in_date; → must return 0

-- 2. Check their views exist
SELECT table_name FROM information_schema.views
WHERE table_schema = 'tembo_hotel' ORDER BY table_name;
→ should show at least 4 views: v_clean_bookings, v_monthly_revenue, v_room_performance etc.
```

```
-- 3. Check their indexes exist
SELECT COUNT(*) FROM pg_indexes WHERE schemaname = 'tembo_hotel';
→ should be at least 8 (7 custom + 1 primary key)

-- 4. Quick revenue sanity check
SELECT SUM(total_amount), COUNT(*) FROM v_clean_bookings;
→ All groups should return very similar numbers (same base dataset)
```

★ Final Instructor Notes

- **Run this whole document yourself on Friday/Saturday before the project starts** — you need a working database so you know what results to expect
- **Each group will get slightly different results** from the dirty data because the CSV has some random variation — but structure should be identical
- **The hardest part for students will be date format cleaning** — especially distinguishing MM-DD-YYYY from DD-MM-YYYY. Be ready to hint: 'look at which part could be > 12'
- **Power BI connection is the second biggest blocker** — test it yourself before Wednesday to have the exact steps ready
- **The best presentations will tell a story**: 'The hotel is growing — revenue increased X% — but suite cancellations are a problem — here's our recommendation'
- **Any group that uses EXISTS vs IN knowingly** or mentions index scan vs seq scan in their presentation deserves bonus recognition — those are advanced insights